

of the function in each event. The aggregation function `count()` increments the number of times the function was called. Other popular aggregation functions include `average()`, `min()`, `max()`, and `sum()`. The probes are created dynamically when the script runs. The probes are disabled automatically when the script stops. DTrace also automatically deallocates any memory it allocated.

The following script uses the `probemod`, `probefunc` table index to collect a table that shows both function name and library name:

```
#!/usr/sbin/dtrace -s
pid$1:::entry
{
    @count_table[probemod,probefunc]=count();
}
```

The library name is in the first column as shown in the following sample:

```
# ./myDscript.d 23
dtrace: script './myDscript' matched 7954 probes
^C
libc.so.1          __clock_gettime      5
libc.so.1          ioctl                 20
libc.so.1          mutex_lock            767
libc.so.1          sigon                 1554
```

5.2.6 Calculating Time Spent in Each Function

To determine how much time is spent in each function, use the DTrace built-in variable `timestamp`. Create probes in the entry and return of each function and then calculate the difference between the two timestamp values. The `timestamp` variable reports time in nanoseconds from epoch.

```
#!/usr/sbin/dtrace -s
pid$1:::entry
{
    ts[probefunc] = timestamp;
}
pid$1:::return
{
    @func_time[probefunc] = sum(timestamp \
        - ts[probefunc]);
    ts[probefunc] = 0;
```